

Solução 1: Botas Perdidas

Solução 1 Vetor de Frequência (Estático) — Funciona pois tamanhos são limitados (≤ 50)

```
MAX = 61
freq_esq = [0] * MAX
freq_dir = [0] * MAX

n = int(input())
for _ in range(n):
    partes = input().split()
    tam = int(partes[0])
    lado = partes[1]
    if lado == 'D':
        freq_dir[tam] += 1
    else:
        freq_esq[tam] += 1

pares = 0
for t in range(1, MAX):
    pares += min(freq_esq[t], freq_dir[t])

print(pares)
```

Solução 2: Botas Perdidas

Solução 2 Dicionário — Hash Map (Dinâmico) — Funciona para qualquer intervalo de tamanhos

```
esq = {}
dir = {}

n = int(input())
for _ in range(n):
    partes = input().split()
    tam = int(partes[0])
    lado = partes[1]
    if lado == 'D':
        dir[tam] = dir.get(tam, 0) + 1
    else:
        esq[tam] = esq.get(tam, 0) + 1

pares = 0
for t in esq:
    pares += min(esq[t], dir[t])

print(pares)
```

Complexidade de Algoritmos



Tabela: Restrições vs Complexidade

Regra de ouro: assuma $\sim 10^8$ operações simples por segundo. Dado o limite de N, deduza a complexidade máxima aceitável.

Limite de N	Complexidade Alvo	Estratégia Típica	Classificação
$N \leq 20$	$O(2^N)$	Força Bruta / Bitmask DP	Exponencial
$N \leq 1.000$	$O(N^2)$	Dois loops aninhados	Quadrático
$N \leq 10^5$	$O(N \log N)$	Sort / Busca Binária	Log-Linear
$N \leq 10^7$	$O(N)$	Frequência / Hash / Um loop	Linear ✓

Exemplo: $O(1)$ — Tempo Constante

$O(1)$ Comparação: loop vs fórmula

```
def soma_lenta(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i  
    return total
```

```
def soma_rapida(n):  
    return n * (n + 1) // 2
```

Exemplo: $O(N)$ — Tempo Linear

$O(N)$ Um único for percorrendo a lista

```
def maior_valor(lista):  
    maior = lista[0]  
  
    for elemento in lista:  
        if elemento > maior:  
            maior = elemento  
  
    return maior
```

Exemplo: $O(N^2)$ — Tempo Quadrático

$O(N^2)$ Dois loops aninhados — todos os pares possíveis

```
def two_product(lista, alvo):  
    n = len(lista)  
  
    for i in range(n):  
        for j in range(i + 1, n):  
            if lista[i] * lista[j] == alvo:  
                return (lista[i], lista[j])  
  
    return None
```

Exemplo: $O(\log N)$ — Tempo Logarítmico

$O(\log N)$ A cada passo, o problema é dividido ao meio

```
def contar_divisoes(n):  
    contagem = 0  
  
    while n > 1:  
        n = n // 2  
        contagem += 1  
  
    return contagem
```

Exemplo: $O(N \log N)$ — Log-Linear

$O(N \log N)$ Para cada elemento da lista, fazemos $O(\log N)$ de trabalho

```
lista = [8, 1024, 3, 64, 1_000_000]

for num in lista:
    n = num
    contagem = 0
    while n > 1:
        n = n // 2
        contagem += 1
    print(f"{num} → {contagem} divisões")
```

Assassinos de Desempenho no Python

Operações que **parecem rápidas** mas escondem um loop interno — invisíveis para quem não conhece, fatais em N grande:

.pop(0)

Remove o **primeiro elemento** da lista. Python precisa **reindexar todos os N-1 elementos** restantes em memória — custoso!

`lista.pop(0) → O(N) ❌`

`deque.popleft() → O(1) ✓`

.remove(x)

Percorre a lista **linearmente** procurando o valor antes de remover. No pior caso, faz N comparações para encontrar o elemento.

`lista.remove(x) → O(N) ❌`

`conjunto.discard(x) → O(1) ✓`

'x' in lista

O operador `in` em listas **examina cada elemento** até encontrar o valor. Pode fazer N comparações por chamada.

`'x' in lista → O(N) ❌`

`'x' in set / dict → O(1) ✓`

O Clássico Two Sum

O Problema

Dado um array de inteiros e um valor **alvo**, encontre os **índices de dois elementos** cuja soma seja exatamente igual ao alvo.

```
nums = [2, 7, 11, 15]
alvo = 9
# → índices (0, 1): 2 + 7 = 9 ✓
```

Two Sum: Força Bruta

$O(N^2)$ Dois loops aninhados

```
def two_sum_bruto(nums, alvo):  
    n = len(nums)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if nums[i] + nums[j] == alvo:  
                return (i, j)  
    return None
```

Two Sum: Solução Ótima

O(N) Dicionário — Lookup em O(1) por complemento

```
def two_sum_otimo(nums, alvo):  
    vistos = {}  
  
    for i, num in enumerate(nums):  
        complemento = alvo - num  
  
        if complemento in vistos:  
            return (vistos[complemento], i)  
  
        vistos[num] = i  
  
    return None
```